PROGRAM: // program for quick sort

```
#include <stdio.h>
// Function to swap two elements
void swap(int* a, int* b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Partition function using the first element (low) as pivot
int partition(int arr[], int low, int high) {
   int pivot = arr[low]; // Select the first element as pivot
   int i = low;
   int j = high;

   while (i < j) {
      // Increment i while elements are less than or equal to pivot
      while (arr[i] <= pivot && i < high) {
         i++;
      }
      // Decrement j while elements are greater than pivot
      while (arr[j] > pivot) {
         j--;
      }
      // If pointers haven't crossed, swap elements
      if (i < j) {
         swap(&arr[i], &arr[j]);
      }
   }

   // Swap the pivot element with the element at index j (its correct position)
   swap(&arr[low], &arr[j]);
   return j; // Return the partition index
}

// Recursive Quick Sort function
void quickSort(int arr[], int low, int high) {
   if (low < high) {
      // Find the partition index
      int pi = partition(arr, low, high);

      quickSort(arr, low, pi - 1);
```

```
      quickSort(arr, pi + 1, high);
   }
}

// Function to print the array
void printArray(int arr[], int size) {
   for (int i = 0; i < size; i++) {
      printf("%d ", arr[i]);
   }
   printf("\n");
}

int main() {
   int arr[] = {24, 9, 29, 14, 19, 4};
   int n = sizeof(arr) / sizeof(arr[0]);

   printf("Original array: ");
   printArray(arr, n);

   quickSort(arr, 0, n - 1);

   printf("Sorted array: ");
   printArray(arr, n);
   return 0;
}
```

OUTPUT:

Original array: 24 9 29 14 19 4

Sorted array: 4 9 14 19 24 29